UT1 Parte 1

Manejo de ficheros

Arquitectura de archivos, clasificación de formatos, APIs de Java (java.io, java.nio.file, java.nio.channels) y gestión de excepciones de E/S

Conceptos fundamentales • Clasificación de formatos • Tratamiento seguro de rutas • Manejo de excepciones

Introducción

Objetivo

Controlar el sistema de ficheros de nuestro equipo desde Java

Es decir ACCEDER A LOS DATOS del sistema de ficheros de nuestro equipo desde Java

Ejemplos: Saber si un determinado fichero está presente en una carpeta, hacer un ls de una carpeta...

Para pensar...

Reflexiona y expresa lo que entiendes por fichero

Busca algunas definiciones "oficiales" de lo que es un fichero

¿Qué representa la clase File?

Concepto de FICHERO

Un fichero (archivo) es una unidad de información almacenada en un dispositivo



La persistencia en disco resuelve la volatilidad de la memoria principal

1. Necesidad de persistencia

- **Memoria RAM volátil:** Los datos de la aplicación se destruyen por completo al finalizar el proceso.
- **Almacenamiento permanente:** Los ficheros permiten mantener información entre ejecuciones en medios físicos.
- **Configuraciones externas:** Permiten modificar el comportamiento del programa sin recompilar código.
- **Intercambio de datos:** Facilitan la comunicación e interoperabilidad entre sistemas distintos.

2. Anatomía de un fichero

- **Contenido:** Datos reales almacenados en el dispositivo de memoria.
- **Metadatos:** Información descriptiva como nombre, tamaño, fechas de creación/modificación, propietario.
- **Permisos:** Reglas de seguridad que determinan lectura (r), escritura (w) o ejecución (x) (Unix, ACLs).
- **Extensión y ruta:** Sufijo que sugiere el formato y dirección lógica de ubicación.
- **Tipos especiales:** Directorios, enlaces simbólicos y archivos ocultos (inician por '.' en Unix).

3. El sistema de archivos

- **Gestión de bloques:** Estructuras como NTFS, ext4 o APFS organizan la asignación física en disco.
- **Diario de cambios (Journaling):** Registra transacciones para recuperar la integridad tras un fallo de energía.
- **Protección y cuotas:** Administra permisos de acceso, cifrado y límites de espacio por usuario.
- **Separadores de ruta:** Usa '\' en sistemas Windows y '/' en sistemas Unix/Linux/macOS.

Clasificación de ficheros

Por tipo de codificación

Texto plano (legible por personas):

Caracteres interpretables según una codificación. El estándar recomendado universalmente es UTF-8.

Ejemplos: código fuente, notas, archivos de configuración, Markdown, CSV.

Archivos binarios (formato máquina):

Información no legible directamente por humanos en texto. Requiere software o códecs específicos para su procesamiento.

Ejemplos: imágenes, audio, vídeo, ejecutables, PDFs, archivos comprimidos.

Por nivel de estructuración

- **Estructurados:** Datos organizados en tablas con esquemas rígidos definidos (ej. filas y columnas en CSV, TSV, Parquet).
- **Semiestructurados:** Información con marcado jerárquico o clave-valor sin tabla fija (ej. JSON para web, XML con validación, YAML para configuraciones, TOML, .properties, Avro, Protobuf).
- **No estructurados:** Contenido libre sin un esquema rígido prediseñado (ej. texto libre en documentos, archivos de audio, fotos, streams de vídeo).

Algunos formatos importantes...

Ofimática y datos tabulares

- **TXT / MD:** Texto plano y Markdown para notas y documentación.
- **PDF:** Maquetación fija universal para distribución e impresión.
- **DOCX / ODT:** Documentos de texto enriquecido modificables.
- **CSV / TSV / XLSX:** Tablas interoperables de datos y hojas de cálculo.

Semiestructurados y configuración

- **JSON / XML:** Intercambio de datos web; XML incluye esquemas de validación.
- **YAML / TOML / INI:** Formatos legibles para archivos de configuración.
- **.properties:** Variables clave-valor estándar en ecosistemas Java.
- **Avro / Protobuf:** Formatos binarios compactos con esquema para servicios.

Imágenes, audio y vídeo

- **Imágenes:** Ráster con pérdida (JPEG), sin pérdida (PNG), GIF; Vectorial (SVG).
- **Audio:** Sin compresión (WAV), sin pérdida (FLAC), con pérdida (MP3, Opus).
- **Vídeo y códecs:** Contenedores (MP4, MKV), códecs (H.264, H.265, AV1), subtítulos (SRT).

Desarrollo, ejecutables y paquetes

- **Código fuente:** Archivos .java, .py, .js, .c, cuadernos Jupyter (.ipynb).
- **Paquetes de software:** Archivos .jar (Java), .whl (Python), .tgz (npm).
- **Ejecutables por SO:** .exe/.dll (Windows PE), ELF (Linux), Mach-O (macOS).
- **Comprimidos y logs:** ZIP, 7z, TAR.GZ; archivos de registro cronológico .log.

Rutas: Indican la localización exacta de un recurso en el sistema de archivos

Concepto y tipos de ruta (path)

- **Definición:** Dirección que guía al sistema operativo hasta un fichero o directorio determinado.
- **Ruta absoluta:** Comienza desde la raíz del sistema (/ en Linux/macOS, C:\ en Windows). Apunta siempre al mismo recurso sin importar desde dónde se ejecute la aplicación.
- **Ruta relativa:** Se evalúa a partir del directorio de trabajo actual desde el que se lanza el programa.
- **Separadores por sistema operativo:** Windows utiliza la barra invertida (\), mientras que los sistemas Unix/Linux emplean la barra inclinada (/).

Esquema de resolución de rutas

```
Estructura en disco:
/ (raíz del sistema)
├── home/
│   └── usuario/
│       └── ejercicios/  <--(Directorio de trabajo)
│           ├── codigo/
│           │   └── HolaMundo.java
│           └── src/

Ejemplos de acceso:
• Ruta absoluta:
/home/usuario/ejercicios/codigo/HolaMundo.java
• Ruta relativa: codigo/HolaMundo.java
```

Java ofrece tres APIs principales según la complejidad y rendimiento requerido

1. java.nio.file (NIO.2)

API moderna recomendada hoy en Java:

- **Path:** Interfaz inmutable para representación de rutas multiplataforma.
- **Files:** Utilidades estáticas para leer, escribir, copiar, mover y atributos.
- **Soporte avanzado:** StandardOpenOption, PathMatcher y WatchService para observar directorios.

2. java.io (API clásica)

Framework tradicional basado en flujos:

- **Flujos binarios:** InputStream y OutputStream para bytes crudos.
- **Flujos de texto:** Reader y Writer gestionando codificación de caracteres.
- **Rendimiento y File:** Buffered* para buffer de memoria y clase tradicional File.

3. java.nio.channels

E/S de alto rendimiento:

- **FileChannel:** Operaciones de canal asíncronas y concurrentes.
- **Bloqueos de archivo:** FileLock para evitar colisiones entre procesos.
- **Memory Mapping:** Proyección directa de ficheros gigantes en la memoria virtual.

Nota de arquitectura en Java: Clases vs. interfaces

- File (java.io): Es una clase concreta tradicional.
- Path (java.nio.file): Es una interfaz inmutable (representa la dirección).
- Files (java.nio.file): Es una clase final de utilidades estáticas que opera sobre objetos Path.

Comparativa java.io.File vs java.nio.file.Path

Operación / Categoría	API clásica: File (java.io)	API moderna: Path y Files (java.nio.file)
Creación de la ruta	`new File(String path)`	`Path.of(...)` ó `Paths.get(...)`
Obtención del nombre	`getName()`	`getFileName()`
Obtención de ruta absoluta	`getAbsolutePath()`	`toAbsolutePath()`
Normalización de ruta	`getCanonicalPath()`	`normalize()` / `toRealPath()`
Creación de directorios	`mkdir()` / `mkdirs()`	`Files.createDirectory()` / `createDirectories()`
Comprobación de tipo y estado	`isFile()` / `isDirectory()` / `exists()`	`Files.isRegularFile()` / `isDirectory()` / `exists()`
Creación de fichero vacío	`createNewFile()`	`Files.createFile(path)`

Ejemplos de lectura, escritura y creación de ficheros

```
// 1. Lectura y escritura rápida en UTF-8
// (NIO.2)
import java.nio.file.*;
import java.nio.charset.StandardCharsets;

public class EjemploNotas {
    public static void main(String[]
args) throws Exception {
        Path file = Path.of("nota.txt");

        // Escribir texto en UTF-8
        Files.writeString(file, "Comprar
pan", StandardCharsets.UTF_8);

        // Leer texto completo
        String texto =
Files.readString(file,
StandardCharsets.UTF_8);
        System.out.println("Contenido: "
+ texto);
    }
}
```

```
// 2. Creación de directorios y ficheros
// Opción A: API moderna (java.nio.file)
Path path = Path.of("docs",
"ejemplo.txt");
Files.createDirectories(path.getParent());
if (Files.notExists(path)) {
    Files.createFile(path);
}

// Opción B: API clásica (java.io.File)
File file = new File("docs/ejemplo.txt");
if (!file.getParentFile().exists()) {
    file.getParentFile().mkdirs();
}
if (!file.exists()) {
    file.createNewFile();
}
```

Manejo de excepciones en el acceso a ficheros

1. Recurso externo

- **Fuera del control de la JVM:** Los ficheros residen en disco o red y no en la memoria interna del programa.
- **Inseguridad en ejecución:** El archivo puede ser borrado, movido o bloqueado por otro proceso externo.
- **Validación previa:** Pueden faltar permisos o no existir las carpetas padre en la ruta dada.

2. Interacción con hardware

- **Fallos físicos:** Desconexión de medios extraíbles, fallos en red o sectores defectuosos.
- **Excepciones comprobadas:** Java marca las operaciones con IOException (el compilador exige try-catch o throws).
- **Tratamiento explícito:** Obliga a definir qué hacer ante la interrupción inesperada de la E/S.

3. try-with-resources

- **Fugas de recursos:** Abrir ficheros consume descriptores de archivo en el sistema operativo.
- **Interfaz AutoCloseable:** Toda clase de E/S implementa esta interfaz para cerrar el flujo automáticamente.
- **Garantía de cierre:** El bloque asegura el cierre incluso si ocurre una excepción inesperada durante la lectura.

Comparativa try – catch vs try-with-resources

```
BufferedReader br = null;
try {
    br =
Files.newBufferedReader(Path.of("datos.t
xt"));
    System.out.println(br.readLine());
} catch (IOException e) {
    System.err.println("Error de
lectura: " + e.getMessage());
} finally {
    // Es obligatorio cerrar manualmente
    // y controlar que no sea null
    if (br != null) {
        try {
            br.close();
        } catch (IOException e) {
            System.err.println("Error al
cerrar: " + e.getMessage());
        }
    }
}
```

```
// El recurso se declara e inicializa
//entre los paréntesis del try
try (BufferedReader br =
Files.newBufferedReader(Path.of("datos.tx
t")))
{
    System.out.println(br.readLine());
}
catch (IOException e)
{
    System.err.println("Error de lectura:
" + e.getMessage());
}
// Al salir del bloque, Java ejecuta
```

Excepciones de E/S según el origen del fallo

Existencia y permisos de acceso

- **NoSuchFileException:** Archivo o ruta no encontrado en disco.
- **FileAlreadyExistsException:** El archivo ya existe al intentar crearlo.
- **AccessDeniedException / SecurityException:** Sin permisos suficientes (ACLs, solo lectura, políticas corporativas).

Espacio y almacenamiento físico

- **IOException (No space left):** Disco físico o partición completamente llena.
- **QuotaExceededException:** Límite de cuota asignada al usuario alcanzado.
- **FileSystemException (Disk full):** Imposibilidad de completar la escritura por falta de espacio.

Concurrencia, bloqueos y rutas

- **OverlappingFileLockException:** El archivo está bloqueado por otro proceso u hilo.
- **IOException (Used by process):** Handle retenido por otra aplicación o proceso zombie.
- **InvalidPathException:** Sintaxis o caracteres no válidos (*, ?, |) en la ruta.

Límites, memoria y hardware

- **OutOfMemoryError:** Lectura completa en RAM de un fichero gigante.
- **IOException (File too large):** Superación de los límites del sistema de archivos.
- **Device not ready / I/O error:** Fallo físico de disco, sector dañado o desconexión de red.

Buenas prácticas en la gestión de ficheros

1. Priorizar java.nio.file (NIO.2): Usar la interfaz Path y la clase de utilidades Files por su inmutabilidad, soporte multiplataforma y riqueza funcional sobre java.io.File.

2. Especificar siempre la codificación (UTF-8): No confiar en la codificación por defecto del sistema operativo al leer o escribir texto plano.

3. Cierre automático de recursos: Emplear try-with-resources en todos los flujos de E/S para liberar descriptores de archivo y evitar fugas de memoria.

4. Captura específica de excepciones: Capturar primero las excepciones concretas (AccessDeniedException, NoSuchFileException) antes de la captura genérica de IOException.

5. Garantizar la portabilidad de rutas: Trabajar con rutas relativas o configurables y emplear Path.of("folder", "file") en lugar de concatenar barras manualmente.